```
"""
╔═══════════════════════════════════════════════════════════════╗
║              NUTRISENSE BOT — ПОВНИЙ КОД                      ║
║              @NutriSenseBot — твій нутриціолог                ║
║                                                               ║
║  Стек:    aiogram 3.x, SQLite, aiohttp                        ║
║  Оплата:  Monobank acquiring (картки Visa/Mastercard)         ║
║  Контент: 28 тижнів освітніх матеріалів                       ║
║  Адмін:   ID 342045533                                        ║
╚═══════════════════════════════════════════════════════════════╝

ВСТАНОВЛЕННЯ:
    pip install aiogram==3.4.1 aiohttp aiosqlite python-dotenv

ЗАПУСК:
    python nutrisense_bot.py

.env файл:
    BOT_TOKEN=your_bot_token
    MONO_TOKEN=your_monobank_token
    ADMIN_ID=342045533
    CHANNEL_ID=@matmatias
    WEBHOOK_URL=https://yourdomain.com/webhook  (якщо webhook)
"""

# ═══════════════════════════════════════════════
# ІМПОРТИ
# ═══════════════════════════════════════════════
import asyncio
import logging
import os
import math
import json
import aiosqlite
import aiohttp
from datetime import datetime, timedelta
from dotenv import load_dotenv

from aiogram import Bot, Dispatcher, F, Router
from aiogram.client.default import DefaultBotProperties
from aiogram.enums import ParseMode
from aiogram.filters import Command, CommandStart, StateFilter
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
```

```
from aiogram.fsm.storage.memory import MemoryStorage
from aiogram.types import (
    Message, CallbackQuery, InlineKeyboardMarkup,
    InlineKeyboardButton, LabeledPrice, PreCheckoutQuery,
    ContentType
)
from aiogram.utils.keyboard import InlineKeyboardBuilder

load_dotenv()
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s"
)
log = logging.getLogger("NutriSense")

# ────────────────────────────────────────────────
# КОНФІГУРАЦІЯ
# ────────────────────────────────────────────────
BOT_TOKEN    = os.getenv("BOT_TOKEN", "")
MONO_TOKEN   = os.getenv("MONO_TOKEN", "")       # Не використовується (jar-оплат
MONO_JAR_URL = "https://send.monobank.ua/jar/8hUo6jMR5M"  # Посилання на банку
ADMIN_ID     = int(os.getenv("ADMIN_ID", "342045533"))
CHANNEL_ID   = os.getenv("CHANNEL_ID", "@matmatias")
WEBHOOK_URL  = os.getenv("WEBHOOK_URL", "")      # порожньо = polling
DB_PATH      = "nutrisense.db"

# Ціни в копійках (UAH * 100)
PLANS = {
    "start": {
        "name":        "Start 🌱",
        "price_uah":   199,
        "price_kopiy": 19900,
        "days":        30,
        "emoji":       "🌱",
        "features": [
            "📊 Точний розрахунок КБЖУ",
            "🍽️ Індивідуальний план харчування",
            "🧠 Тест харчової поведінки",
            "📚 28 тижнів освітнього контенту",
            "📋 Рекомендації по продуктах",
        ]
    },
    "premium": {
        "name":        "Premium 🌿",
        "price_uah":   349,
        "price_kopiy": 34900,
        "days":        30,
```

```
        "emoji":       "🌿",
        "features": [
            "✅ Все з Start",
            "📸 Аналіз фото тарілки",
            "📅 Щоденний трекер їжі",
            "📈 Щотижневий звіт прогресу",
            "💬 Пріоритетна підтримка",
        ]
    },
    "vip": {
        "name":        "VIP 👑",
        "price_uah":   549,
        "price_kopiy": 54900,
        "days":        30,
        "emoji":       "👑",
        "features": [
            "✅ Все з Premium",
            "🤖 ШІ-нутриціолог (необмежено)",
            "🎯 Персональний план на місяць",
            "⚡ Відповідь протягом 1 години",
            "🏆 VIP-чат з нутриціологом",
        ]
    }
}

# ──────────────────────────────────────────────
# 28 ТИЖНІВ КОНТЕНТУ
# Структура: тиждень → {тема, пости[]}
# Кожен пост: {день, рубрика, текст}
# ──────────────────────────────────────────────
WEEKLY_CONTENT = {
    1: {
        "theme": "🌱 Основи усвідомленого харчування",
        "posts": [
            {
                "day": 1,
                "rubric": "знання",
                "title": "Що таке усвідомлене харчування?",
                "text": (
                    "🧠 <b>Тиждень 1 · День 1</b>\n\n"
                    "<b>Усвідомлене харчування — це не дієта.</b>\n\n"
                    "Це вміння чути своє тіло:\n"
                    "→ їсти, коли є фізичний голод\n"
                    "→ зупинятися при насиченні\n"
                    "→ обирати їжу, яка дає енергію і задоволення\n\n"
                    "Формула здорової харчової поведінки:\n"
                    "<code>Їм за фізіологічним голодом +\n"
```

```
            "Смачну та поживну їжу +\n"
            "Зупиняюся за насиченням +\n"
            "Забуваю про їжу до наступного голоду</code>\n\n"
            "Цього тижня навчимось розрізняти голод 🌿"
        )
    },
    {
        "day": 2,
        "rubric": "поведінка",
        "title": "Шкала голоду-ситості",
        "text": (
            "🩸 <b>Тиждень 1 · День 2</b>\n\n"
            "<b>Шкала голоду-ситості (1–10)</b>\n\n"
            "1️⃣ — Дуже сильний голод, паморочиться голова\n"
            "2️⃣ — Комфортний голод, треба їсти\n"
            "3️⃣ — Відчуваю легкий голод\n"
            "4️⃣ — Нейтральний стан ← <b>починай їсти тут</b>\n"
            "5️⃣ — Легка ситість\n"
            "6️⃣ — Комфортна ситість ← <b>зупиняйся тут</b>\n"
            "7️⃣ — Насичення, варто зупинитись\n"
            "8️⃣ — Трішки переїв\n"
            "9️⃣ — Дуже переїв\n"
            "🔟 — Крайня ситість\n\n"
            "<b>Практика:</b> перед кожним прийомом їжі сьогодні — переві
        )
    },
    {
        "day": 4,
        "rubric": "тарілка",
        "title": "Ідеальна тарілка",
        "text": (
            "🍽️ <b>Тиждень 1 · День 4</b>\n\n"
            "<b>Формула збалансованої тарілки:</b>\n\n"
            "🥗 <b>½ тарілки</b> — різнокольорові овочі\n"
            "🍗 <b>¼ тарілки</b> — білок\n"
            "🌾 <b>¼ тарілки</b> — складні вуглеводи\n"
            "🫒 <b>невелика кількість</b> — корисні жири\n\n"
            "Ця формула працює для сніданку, обіду і вечері.\n"
            "Сьогодні склади тарілку за цією схемою і поділись в коментар
        )
    },
    {
        "day": 6,
        "rubric": "знання",
        "title": "Чому заборони не працюють",
        "text": (
            "🚫 <b>Тиждень 1 · День 6</b>\n\n"
```

```
                "<b>Немає поганої чи забороненої їжі.</b>\n\n"
                "Коли ми забороняємо собі щось:\n"
                "❌ мозок фіксується на забороненому\n"
                "❌ зростає тяга і напруга\n"
                "❌ зривається «сила волі» → відчуття провини\n"
                "❌ цикл повторюється\n\n"
                "Коли їжа дозволена:\n"
                "✅ зникають зриви\n"
                "✅ баланс стає важливішим за ідеальність\n"
                "✅ повертається задоволення від їжі\n\n"
                "Мета — не ідеальне харчування, а стабільне 🌿"
            )
        },
    ]
},
2: {
    "theme": "🧬 Макронутрієнти: Білок",
    "posts": [
        {
            "day": 1,
            "rubric": "знання",
            "title": "Навіщо нам білок?",
            "text": (
                "💪 <b>Тиждень 2 · День 1</b>\n\n"
                "<b>Білок — будівельний матеріал тіла.</b>\n\n"
                "Дефіцит білка:\n"
                "→ випадіння волосся та ламкість нігтів\n"
                "→ збої в гормональній системі\n"
                "→ ризик набряків\n"
                "→ зниження імунітету\n\n"
                "Важлива деталь: якщо в раціоні мало вуглеводів,\n"
                "організм використовує білок як паливо — і тіло страждає.\n\n
                "<b>Норма білка:</b> 1.4-2.0 г на кг ваги тіла\n"
                "Розрахуй точно через /kbju 📊"
            )
        },
        {
            "day": 3,
            "rubric": "тарілка",
            "title": "Жирний vs нежирний білок",
            "text": (
                "🐟 <b>Тиждень 2 · День 3</b>\n\n"
                "<b>ЖИРНИЙ БІЛОК</b>\n"
                "• Жирна риба: лосось, форель, скумбрія\n"
                "• Цілі яйця\n"
                "• Червоне м'ясо: свинина, баранина, качка\n"
                "• Домашній сир 9%\n"
```

```
                "• Тофу та темпе\n\n"
                "<b>НЕЖИРНИЙ БІЛОК</b>\n"
                "• Куряче філе та філе індички (без шкіри)\n"
                "• Яєчний білок (без жовтка)\n"
                "• Біла риба: тріска, хек, судак\n"
                "• Морепродукти: креветки, кальмари\n"
                "• Знежирений сир 2%\n\n"
                "<b>Порада:</b> на сніданку — жирний, на вечерю — нежирний 🌿
            )
        },
        {
            "day": 5,
            "rubric": "рецепт",
            "title": "Сніданок з білком за 10 хв",
            "text": (
                "🥘 <b>Тиждень 2 · День 5</b>\n\n"
                "<b>Омлет з овочами та сиром</b>\n\n"
                "Інгредієнти:\n"
                "• 3 яйця\n"
                "• 50г шпинату або рукколи\n"
                "• ½ помідора\n"
                "• 30г твердого сиру\n"
                "• 1 ч.л. оливкової олії\n\n"
                "Приготування: 10 хвилин\n\n"
                "📊 КБЖУ на порцію:\n"
                "<code>Калорії:   320 ккал\n"
                "Білок:     24 г\n"
                "Жири:      22 г\n"
                "Вуглеводи:  4 г</code>\n\n"
                "Ідеальний старт дня з повним почуттям ситості до обіду 💚"
            )
        },
    ]
},
3: {
    "theme": "🧈 Макронутрієнти: Жири",
    "posts": [
        {
            "day": 1,
            "rubric": "знання",
            "title": "Жири — не ворог",
            "text": (
                "🥑 <b>Тиждень 3 · День 1</b>\n\n"
                "<b>Жири — основа для гормонального здоров'я.</b>\n\n"
                "Дефіцит жирів:\n"
                "→ гормональний збій (статеві гормони синтезуються з холестер
                "→ зниження лібідо, гормональний дисбаланс\n"
```

```
                "→ суха шкіра, ламке волосся\n"
                "→ вітаміни A, D, E, K не засвоюються без жирів\n"
                "→ мозок на 60% з жирів — без них гірше думається\n\n"
                "Надлишок насичених жирів і трансжирів:\n"
                "→ серцево-судинні ризики\n"
                "→ метаболічні порушення\n\n"
                "<b>Золоте правило:</b> жири потрібні, але правильні 🌿"
            )
        },
        {
            "day": 3,
            "rubric": "тарілка",
            "title": "Насичені / Ненасичені / Омега-3",
            "text": (
                "🥑 <b>Тиждень 3 · День 3</b>\n\n"
                "<b>НАСИЧЕНІ ЖИРИ</b> (обмежити)\n"
                "• Вершкове масло, сало\n"
                "• Всі молочні продукти\n"
                "• Жирне м'ясо, шкіра птиці\n"
                "• Пальмова та кокосова олія\n\n"
                "<b>НЕНАСИЧЕНІ ЖИРИ</b> ✅\n"
                "• Оливкова олія\n"
                "• Авокадо, маслини\n"
                "• Мигдаль, фундук, кеш'ю\n"
                "• Арахіс, ріпакова олія\n\n"
                "<b>ОМЕГА-3</b> ✅✅\n"
                "• Лосось, скумбрія, оселедець\n"
                "• Насіння льону, чіа\n"
                "• Волоські горіхи\n"
                "• Конопляна олія\n\n"
                "Омега-3 — 2-3 рази на тиждень обов'язково 💚"
            )
        },
    ]
},
4: {
    "theme": "🌾 Макронутрієнти: Вуглеводи",
    "posts": [
        {
            "day": 1,
            "rubric": "знання",
            "title": "Вуглеводи — паливо для мозку",
            "text": (
                "⚡ <b>Тиждень 4 · День 1</b>\n\n"
                "<b>Вуглеводи — головне паливо для мозку та м'язів.</b>\n\n"
                "При нестачі вуглеводів:\n"
                "→ «Мозковий туман» — важко концентруватись\n"
```

```
            "→ Дратівливість, апатія, біль у м'язах\n"
            "→ Порушення нервової системи\n\n"
            "При надлишку вуглеводів:\n"
            "→ Трансформація в жир через інсулін\n"
            "→ Інсулінорезистентність\n"
            "→ Ризик цукрового діабету 2 типу\n\n"
            "<b>Золоте правило:</b>\n"
            "Їжте переважно складні вуглеводи — крупи, цільнозерновий хлі
            "Вони дають тривалу енергію без стрибків цукру 🌿"
        )
    },
    {
        "day": 3,
        "rubric": "тарілка",
        "title": "Прості vs складні вуглеводи",
        "text": (
            "🌾 <b>Тиждень 4 · День 3</b>\n\n"
            "<b>ПРОСТІ ВУГЛЕВОДИ</b> (обмежити)\n"
            "Фрукти, сухофрукти, ягоди,\n"
            "Цукор, мед, солодкі сиропи,\n"
            "Солодощі, солодкі напої, йогурти\n\n"
            "<b>СКЛАДНІ ВУГЛЕВОДИ</b> ✅ (основа)\n"
            "• Крупи: гречка, овес, пшоно, булгур\n"
            "• ЦЗ хліб (цільнозерновий)\n"
            "• Бобові: сочевиця, нут, квасоля\n"
            "• Паста ТСП (тверді сорти пшениці)\n"
            "• Кукурудза, батат, картопля\n\n"
            "<b>Порада:</b> чим більше клітковини — тим повільніше засвою
            "тим довше ситість і стабільний цукор 💚"
        )
    },
    {
        "day": 5,
        "rubric": "знання",
        "title": "Овочі — суперфуд кожного дня",
        "text": (
            "🥦 <b>Тиждень 4 · День 5</b>\n\n"
            "<b>Овочі — головне джерело клітковини, вітамінів і антиоксид
            "Чому вони важливі:\n"
            "🌿 Клітковина — «щітка» для кишківника\n"
            "🌿 Великий об’єм при низькій калорійності → ситість\n"
            "🌿 Вітаміни очищають судини та захищають клітини\n\n"
            "<b>Правило «веселки»:</b>\n"
            "🟢 Зелені: шпинат, броколі, огірок, кабачок\n"
            "🔴 Червоні: помідори, перець, буряк\n"
            "🟡 Жовті: кукурудза, гарбуз, морква\n"
            "🟣 Фіолетові: капуста, баклажан, цибуля\n\n"
```

```
                "Прагни до мінімум ½ тарілки овочів при кожному прийомі їжі 🎯
            )
        },
    ]
},
5: {
    "theme": "🧠 Харчова поведінка",
    "posts": [
        {
            "day": 1,
            "rubric": "поведінка",
            "title": "3 типи голоду",
            "text": (
                "🧠 <b>Тиждень 5 · День 1</b>\n\n"
                "<b>Гомеостатичний голод</b> 🟢\n"
                "Наростає поступово, фізичні сигнали.\n"
                "Що робити: <i>їсти</i>\n\n"
                "<b>Гедонічний голод</b> 🟡\n"
                "Виникає без фізичного голоду, хочеться щось смачне.\n"
                "Може виникати від нудьги, стресу, порожнечі.\n"
                "Що робити: зробити паузу, відволіктись, і якщо все ще хочеш
                "<b>Дисрегульований голод</b> 🔴\n"
                "Голод без насичення, не можу зупинитись.\n"
                "Що робити: регулярні прийоми їжі,\n"
                "достатня калорійність, сон, зниження стресу 🌿"
            )
        },
        {
            "day": 3,
            "rubric": "поведінка",
            "title": "Як впоратись з гедонічним голодом",
            "text": (
                "💡 <b>Тиждень 5 · День 3</b>\n\n"
                "<b>Що робити з емоційним голодом?</b>\n\n"
                "<b>Якщо вам НУДНО:</b>\n"
                "• Вийди на вулицю на 10 хвилин\n"
                "• Малювання, гра, 4 музичні інструменти\n"
                "• Склади пазл, займись рукоділлям\n\n"
                "<b>Якщо ви СТРЕСУЄТЕ:</b>\n"
                "• Дихання 4-7-8\n"
                "• Контакт з водою: тепла ванна, миття посуду\n"
                "• Важка ковдра: 15 хвилин під нею\n\n"
                "<b>Якщо ви САМОТНІ:</b>\n"
                "• Голосові повідомлення друзям\n"
                "• Домашні тварини (погладь кота)\n"
                "• Подкаст-розмова\n\n"
                "<b>Якщо ви ВТОМИЛИСЬ:</b>\n"
```

```
                "• Power Nap 15-20 хвилин\n"
                "• Свіже повітря\n"
                "• Музичний драйв 🎵"
            )
        },
    ]
},
6: {
    "theme": "😴 Сон, стрес і харчування",
    "posts": [
        {
            "day": 1,
            "rubric": "знання",
            "title": "Як сон впливає на вагу",
            "text": (
                "😴 <b>Тиждень 6 · День 1</b>\n\n"
                "<b>Сон регулює відчуття голоду і насичення.</b>\n\n"
                "При нестачі сну:\n"
                "→ Недостатньо підтискає тягу та знижує контроль\n"
                "→ Зростає рівень греліну (гормон голоду)\n"
                "→ Знижується лептин (гормон насичення)\n"
                "→ Тягне на солодке і жирне\n\n"
                "Що робити:\n"
                "✅ Стабільний час сну й пробудження\n"
                "✅ 7-9 годин сну\n"
                "✅ Вечеря не на біг і не перед самим сном\n"
                "✅ Мінімізувати екрани перед сном 🌿"
            )
        },
        {
            "day": 3,
            "rubric": "знання",
            "title": "Стрес-їжа: чому це відбувається",
            "text": (
                "😰 <b>Тиждень 6 · День 3</b>\n\n"
                "<b>Стрес підвищує потребу в швидкому заспокоєнні.</b>\n\n"
                "Що відбувається:\n"
                "→ Кортизол → тяга до солодкого і жирного\n"
                "→ Їжа стає інструментом регуляції емоцій\n"
                "→ Проблема у відсутності альтернативи\n\n"
                "Як допомогти собі:\n"
                "→ Дихання, рух, контакт, пауза\n"
                "→ Не забороняти їжу, а розширювати вибір реакцій\n\n"
                "<b>Відновлення — це не слабкість.\n"
                "Це база здорової харчової поведінки.</b> 💚"
            )
        },
```

```
    ]
},
7: {
    "theme": "🦠 Пробіотики і мікробіом",
    "posts": [
        {
            "day": 1,
            "rubric": "знання",
            "title": "Твій мікробіом керує голодом",
            "text": (
                "🦠 <b>Тиждень 7 · День 1</b>\n\n"
                "<b>Мікробіом — 90% сероніту синтезується в кишківнику.</b>\n
                "Здорові бактерії впливають на:\n"
                "🧠 Настрій і нервова система\n"
                "🛡️ Імунітет (перша лінія захисту)\n"
                "🍽️ Травлення і засвоєння нутрієнтів\n"
                "🎭 Апетит і тягу: дисбаланс мікробіому пов'язаний\n"
                "з переїданням і тягою до солодкого\n\n"
                "Баланс і гормональне здоров'я:\n"
                "Здорові бактерії знижують системне запалення\n"
                "і підтримують гормони 💚"
            )
        },
        {
            "day": 3,
            "rubric": "тарілка",
            "title": "Пробіотики і пребіотики",
            "text": (
                "🫚 <b>Тиждень 7 · День 3</b>\n\n"
                "<b>ПРОБІОТИКИ</b> (корисні бактерії):\n"
                "• Натуральний йогурт без цукру з живими культурами\n"
                "• Кефір (кисломолочний напій)\n"
                "• Квашена капуста (саме ферментована)\n"
                "• Кімчі, Комбуча, Місо\n"
                "• Темпе, натуральний яблучний оцет\n\n"
                "<b>ПРЕБІОТИКИ</b> (їжа для бактерій):\n"
                "• Цільнозернові крупи, сочевиця\n"
                "• Насіння льону, какао (без цукру)\n"
                "• Часник, цибуля, цикорій\n"
                "• Яблука, банани, ягоди\n"
                "• Ферментовані продукти щодня 🌿"
            )
        },
    ]
},
8: {
    "theme": "🍽️ Сніданок — найважливіший прийом?",
```

```
        "posts": [
            {
                "day": 1,
                "rubric": "знання",
                "title": "Сніданок: правила балансу",
                "text": (
                    "🌅 <b>Тиждень 8 · День 1</b>\n\n"
                    "<b>Ідеальна тарілка сніданку:</b>\n\n"
                    "🥩 <b>Білок</b> — ¼ тарілки\n"
                    "Жирні та нежирні джерела\n\n"
                    "🌾 <b>Вуглеводи</b> — ⅙ тарілки\n"
                    "Найменша кількість складних вуглеводів\n"
                    "(можна додати десерт за бажанням)\n\n"
                    "🥑 <b>Жири насичені</b> — ¼ тарілки\n"
                    "Це прийом, найбільш збагачений жирами\n\n"
                    "🫒 <b>Жири ненасичені</b> — ⅙ тарілки\n\n"
                    "🍓 <b>Овочі та ягоди</b> — ½ тарілки\n"
                    "Різнокольорові, у будь-якому вигляді 💚"
                )
            },
            {
                "day": 3,
                "rubric": "рецепт",
                "title": "5 ідеальних сніданків",
                "text": (
                    "🍳 <b>Тиждень 8 · День 3</b>\n\n"
                    "<b>5 збалансованих сніданків:</b>\n\n"
                    "1️⃣ Овсянка + ягоди + горіхи + яйце\n"
                    "2️⃣ Омлет + авокадо + цільнозерновий хліб\n"
                    "3️⃣ Йогурт без цукру + фрукти + насіння + горіхи\n"
                    "4️⃣ Яйця пашот + шпинат + лосось + огірок\n"
                    "5️⃣ Гречана каша + яйце круто + овочевий салат\n\n"
                    "Кожен містить: білок + жири + вуглеводи + клітковину.\n"
                    "Ситість до обіду гарантована 🌿"
                )
            },
        ]
    },
    9: {
        "theme": "☀️ Обід — центр дня",
        "posts": [
            {
                "day": 1,
                "rubric": "тарілка",
                "title": "Формула обіду",
                "text": (
                    "☀️ <b>Тиждень 9 · День 1</b>\n\n"
```

```
                "<b>Ідеальна тарілка обіду:</b>\n\n"
                "🥩 <b>Білок</b> — ¼ тарілки\n"
                "Жирні та нежирні джерела\n\n"
                "🌾 <b>Вуглеводи</b> — ¼ тарілки\n"
                "Найбільша кількість складних вуглеводів\n"
                "серед усіх прийомів (можна додати десерт)\n\n"
                "🫒 <b>Жири</b> — невелика кількість\n"
                "Олія в каші, в салаті — не більше\n\n"
                "🥗 <b>Овочі</b> — ½ тарілки різнокольорових 💚"
            )
        },
    ]
},
10: {
    "theme": "🌙 Вечеря — легка і поживна",
    "posts": [
        {
            "day": 1,
            "rubric": "тарілка",
            "title": "Формула вечері",
            "text": (
                "🌙 <b>Тиждень 10 · День 1</b>\n\n"
                "<b>Ідеальна тарілка вечері:</b>\n\n"
                "🥩 <b>Білок</b> — ⅓ тарілки\n"
                "Краще обрати нежирні джерела білку\n\n"
                "🌾 <b>Вуглеводи</b> — ⅙ тарілки\n"
                "Лише складні джерела, менша кількість\n\n"
                "🫒 <b>Жири</b> — найменша кількість\n\n"
                "🥗 <b>Овочі</b> — ½ тарілки різнокольорових 🌿\n\n"
                "<b>Типові помилки вечері:</b>\n"
                "❌ Переважно жирний білок\n"
                "❌ Профіцит жирів\n"
                "❌ Відсутність круп\n"
                "❌ Одноманітні джерела вуглеводів"
            )
        },
    ]
},
# Тижні 11-28: продовження контенту
11: {
    "theme": "💧 Гідратація та харчування",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Вода та обмін речовин",
            "text": (
                "💧 <b>Тиждень 11 · День 1</b>\n\n"
```

```
                "<b>Вода — основа всіх метаболічних процесів.</b>\n\n"
                "Норма: <b>30-35 мл на кг ваги</b>\n\n"
                "Ознаки нестачі:\n"
                "→ Втома і «туман» у голові\n"
                "→ Хибне відчуття голоду\n"
                "→ Запори і погане травлення\n"
                "→ Суха шкіра\n\n"
                "Лайфхак: перед кожним прийомом їжі\n"
                "випий 1 склянку води — це зменшить порцію\n"
                "і покращить травлення 💚"
            )
        },
    ]
},
12: {
    "theme": "🏋️ Харчування та фізична активність",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Їжа до та після тренування",
            "text": (
                "🏋️ <b>Тиждень 12 · День 1</b>\n\n"
                "<b>До тренування (за 1-2 год):</b>\n"
                "🌾 Складні вуглеводи + трохи білку\n"
                "Овсянка з бананом, рисова каша + яйце\n\n"
                "<b>Після тренування (протягом 30-60 хв):</b>\n"
                "🥩 Білок + прості вуглеводи\n"
                "Курка + рис, йогурт + фрукти\n\n"
                "<b>Навіщо вуглеводи після тренування?</b>\n"
                "Відновлюють глікоген у м'язах\n"
                "і допомагають білку будувати, а не горіти 💪"
            )
        },
    ]
},
13: {
    "theme": "🫐 Клітковина і травлення",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Норма клітковини на день",
            "text": (
                "🌿 <b>Тиждень 13 · День 1</b>\n\n"
                "<b>Норма клітковини: 25-35 г на день.</b>\n\n"
                "Більшість людей отримують лише 10-15 г.\n\n"
                "Джерела клітковини:\n"
                "🥦 Овочі і зелень\n"
```

```
                "🍎 Фрукти (з шкіркою)\n"
                "🌾 Цільнозернові крупи\n"
                "🫘 Бобові\n"
                "🌰 Горіхи та насіння\n\n"
                "Клітковина:\n"
                "✅ Годує корисні бактерії\n"
                "✅ Знижує рівень цукру крові\n"
                "✅ Знижує холестерин\n"
                "✅ Дає тривалу ситість 💚"
            )
        },
    ]
},
14: {
    "theme": "🍫 Як їсти солодке без шкоди",
    "posts": [
        {
            "day": 1, "rubric": "поведінка",
            "title": "Десерт без провини",
            "text": (
                "🍫 <b>Тиждень 14 · День 1</b>\n\n"
                "<b>Солодке можна. Питання — як і коли.</b>\n\n"
                "Найкращий час для десерту:\n"
                "✅ Після сніданку або обіду (не вечері)\n"
                "✅ Після повноцінного прийому їжі\n"
                "✅ Коли є фізичний голод, а не емоційний\n\n"
                "Що робить десерт «безпечним»:\n"
                "→ Є після їжі, а не замість\n"
                "→ Порція, яка задовольняє, а не «поки не закінчиться»\n"
                "→ Повне усвідомлене задоволення, без почуття провини\n\n"
                "Заборона → зрив → провина → заборона\n"
                "Дозвіл → задоволення → контроль 💚"
            )
        },
    ]
},
15: {
    "theme": "⚖️ Управління вагою без дієт",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Дефіцит калорій: як правильно",
            "text": (
                "⚖️ <b>Тиждень 15 · День 1</b>\n\n"
                "<b>Безпечний дефіцит: 300–500 ккал від норми.</b>\n\n"
                "Менше — тіло адаптується і ефект зникає.\n"
                "Більше — тіло втрачає м'язи, гормони страждають.\n\n"
```

```
                "<b>Правила безпечного зниження ваги:</b>\n"
                "✅ Не менше 1200-1500 ккал (залежно від ваги і статі)\n"
                "✅ Достатньо білку (1.6-2 г/кг)\n"
                "✅ Тренування зі збереженням м'язів\n"
                "✅ Темп: 0.5-1 кг на тиждень — ідеально\n\n"
                "Розрахуй свою норму через /kbju 📊"
            )
        },
    ]
},
16: {
    "theme": "⚖️ Гормони та харчування",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Як харчування впливає на гормони",
            "text": (
                "⚖️ <b>Тиждень 16 · День 1</b>\n\n"
                "<b>Гормони керують апетитом, настроєм і вагою.</b>\n\n"
                "🟢 <b>Інсулін</b> — регулює цукор в крові.\n"
                "Надлишок простих вуглеводів → стрибки інсуліну\n"
                "→ жировідкладення і постійний голод\n\n"
                "🟡 <b>Лептин</b> — сигнал ситості.\n"
                "При дефіциті сну і калорій знижується\n"
                "→ мозок не отримує сигнал «я ситий»\n\n"
                "🔴 <b>Кортизол</b> — гормон стресу.\n"
                "Хронічний стрес → тяга до солодкого і жирного\n"
                "→ накопичення жиру на животі\n\n"
                "<b>Як підтримати баланс:</b>\n"
                "✅ Збалансована тарілка\n"
                "✅ Регулярні прийоми їжі\n"
                "✅ Якісний сон 7-9 год\n"
                "✅ Зниження стресу 🌿"
            )
        },
        {
            "day": 3, "rubric": "знання",
            "title": "Щитовидна залоза і вага",
            "text": (
                "🦋 <b>Тиждень 16 · День 3</b>\n\n"
                "<b>Щитовидна залоза керує метаболізмом.</b>\n\n"
                "Гіпотиреоз (знижена функція):\n"
                "→ Уповільнений метаболізм\n"
                "→ Набір ваги без зміни раціону\n"
                "→ Втома, мерзлякуватість, набряки\n\n"
                "Що підтримує здоров'я щитовидки:\n"
                "🧂 Йод: морепродукти, йодована сіль\n"
```

```
                "🌰 Селен: бразильський горіх (1-2 шт/день)\n"
                "🥜 Цинк: гарбузове насіння, м'ясо\n"
                "🐟 Омега-3: жирна риба\n\n"
                "⚠️ Здай аналіз ТТГ раз на рік —\n"
                "проста і важлива перевірка 💚"
            )
        },
    ]
},
17: {
    "theme": "🔢 Мікронутрієнти: Вітаміни",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Вітамін D: чому всі мають дефіцит",
            "text": (
                "☀️ <b>Тиждень 17 · День 1</b>\n\n"
                "<b>Вітамін D — гормон, а не просто вітамін.</b>\n\n"
                "Дефіцит D (є у більшості українців):\n"
                "→ Втома, депресія, тривога\n"
                "→ Слабкий імунітет\n"
                "→ Ламкі кістки\n"
                "→ Порушення гормонального балансу\n\n"
                "Джерела:\n"
                "☀️ Сонце (основне джерело)\n"
                "🐟 Жирна риба (лосось, скумбрія)\n"
                "🥚 Жовтки яєць\n"
                "🫙 Добавки D3 (рекомендую здати аналіз і підібрати дозу)\n\n
                "Оптимальний рівень: 50-80 нг/мл 💚"
            )
        },
    ]
},
18: {
    "theme": "💊 Мікронутрієнти: Мінерали",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Залізо: коли їжа не допомагає",
            "text": (
                "🩸 <b>Тиждень 18 · День 1</b>\n\n"
                "<b>Залізодефіцитна анемія — поширена проблема.</b>\n\n"
                "Ознаки дефіциту:\n"
                "→ Хронічна втома\n"
                "→ Блідість шкіри\n"
                "→ Випадіння волосся\n"
                "→ Тахікардія при навантаженні\n\n"
```

```
                "Джерела заліза:\n"
                "🥩 Гемове (засвоюється краще): червоне м'ясо, печінка\n"
                "🌿 Негемове: шпинат, гречка, сочевиця\n\n"
                "<b>Лайфхак:</b> їж залізо з вітаміном C\n"
                "(лимон, перець) — засвоєння зростає на 50%\n"
                "Не пий каву 1 год до і після їжі 🌿"
            )
        },
    ]
},
19: {
    "theme": "🧘 Стрес і кортизол",
    "posts": [
        {
            "day": 1, "rubric": "поведінка",
            "title": "Кортизол і вага — прямий зв'язок",
            "text": (
                "😤 <b>Тиждень 19 · День 1</b>\n\n"
                "<b>Хронічний стрес = хронічний кортизол.</b>\n\n"
                "Кортизол:\n"
                "→ Підвищує апетит\n"
                "→ Підсилює тягу до солодкого і жирного\n"
                "→ Сприяє накопиченню жиру на животі\n"
                "→ Руйнує м'язову тканину\n\n"
                "<b>Що знижує кортизол:</b>\n"
                "✅ Якісний сон 7-9 годин\n"
                "✅ Фізична активність (але не надмірна!)\n"
                "✅ Медитація та дихальні практики\n"
                "✅ Спілкування з близькими\n"
                "✅ Природа та прогулянки\n"
                "✅ Адаптогени: ашваганда, родіола 🌿"
            )
        },
    ]
},
20: {
    "theme": "🍱 Meal Prep — готовлю раз на тиждень",
    "posts": [
        {
            "day": 1, "rubric": "практика",
            "title": "Як організувати meal prep",
            "text": (
                "📦 <b>Тиждень 20 · День 1</b>\n\n"
                "<b>Meal Prep — готовка на тиждень за 2 години.</b>\n\n"
                "Що готувати заздалегідь:\n"
                "🌾 Крупи: гречка, рис, булгур (3-5 днів)\n"
                "🥩 Білок: запечена курка, яйця, риба\n"
```

```
                "🥗 Нарізані овочі в контейнерах\n"
                "🫘 Відварена сочевиця або нут\n\n"
                "Алгоритм sunday meal prep:\n"
                "1️⃣ Відвари 3 крупи\n"
                "2️⃣ Запечи білок (курка + риба)\n"
                "3️⃣ Наріж овочі по контейнерах\n"
                "4️⃣ Зроби 1-2 соуси\n\n"
                "Результат: 5 днів без думок «що приготувати» 💚"
            )
        },
    ]
},
21: {
    "theme": "🎯 Постановка харчових цілей",
    "posts": [
        {
            "day": 1, "rubric": "поведінка",
            "title": "SMART-цілі у харчуванні",
            "text": (
                "🎯 <b>Тиждень 21 · День 1</b>\n\n"
                "<b>Чому «їсти здоровіше» — не ціль.</b>\n\n"
                "Правильна ціль за SMART:\n"
                "✅ <b>Specific</b> — конкретна\n"
                "✅ <b>Measurable</b> — вимірювана\n"
                "✅ <b>Achievable</b> — досяжна\n"
                "✅ <b>Relevant</b> — важлива особисто\n"
                "✅ <b>Time-bound</b> — з терміном\n\n"
                "Приклад:\n"
                "❌ «Хочу харчуватися здоровіше»\n"
                "✅ «Протягом 4 тижнів їстиму\n"
                "5 порцій овочів кожного дня\n"
                "і готуватиму сніданок вдома»\n\n"
                "Маленькі конкретні кроки = великий результат 🌿"
            )
        },
    ]
},
22: {
    "theme": "🧪 Аналізи та харчування",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Які аналізи здати для контролю здоров'я",
            "text": (
                "🧪 <b>Тиждень 22 · День 1</b>\n\n"
                "<b>Базові аналізи для контролю здоров'я:</b>\n\n"
                "🩸 <b>Загальний аналіз крові</b>\n"
```

```
                "Гемоглобін, феритин, залізо\n\n"
                "🧫 <b>Біохімія</b>\n"
                "Глюкоза, холестерин, тригліцериди\n\n"
                "🦋 <b>Щитовидна залоза</b>\n"
                "ТТГ, Т3, Т4 — впливає на вагу та метаболізм\n\n"
                "☀️ <b>Вітамін D, B12</b>\n"
                "Дефіцит є у більшості\n\n"
                "🥦 <b>Магній та цинк</b>\n"
                "Критичні для гормонів і нервової системи\n\n"
                "Здавай раз на 6 місяців 💚"
            )
        },
    ]
},
23: {
    "theme": "🌍 Харчування в дорозі та кафе",
    "posts": [
        {
            "day": 1, "rubric": "практика",
            "title": "Як їсти правильно поза домом",
            "text": (
                "🍽️ <b>Тиждень 23 · День 1</b>\n\n"
                "<b>Кафе і ресторани — не ворог.</b>\n\n"
                "Як обирати:\n"
                "✅ Шукай білок в меню (м'ясо, риба, бобові)\n"
                "✅ Проси соус окремо\n"
                "✅ Замінюй картоплю фрі на овочі\n"
                "✅ Замовляй воду, а не солодкий напій\n"
                "✅ Їж повільно і насолоджуйся\n\n"
                "<b>В дорозі (закуски):</b>\n"
                "🥜 Горіхи (мигдаль, кеш'ю)\n"
                "🧀 Твердий сир у нарізці\n"
                "🍎 Фрукти або овочі\n"
                "🥚 Варені яйця\n"
                "🍫 Темний шоколад 70%+ 🌿"
            )
        },
    ]
},
24: {
    "theme": "📱 Трекінг їжі: за і проти",
    "posts": [
        {
            "day": 1, "rubric": "поведінка",
            "title": "Рахувати калорії чи ні?",
            "text": (
                "📱 <b>Тиждень 24 · День 1</b>\n\n"
```

```
                "<b>Трекінг їжі — інструмент, не тюрма.</b>\n\n"
                "<b>Коли рахувати корисно:</b>\n"
                "✅ Хочеш зрозуміти свій раціон\n"
                "✅ Не виходить результат при правильному харчуванні\n"
                "✅ Новачок у нутриціології\n\n"
                "<b>Коли рахувати шкідливо:</b>\n"
                "❌ Є схильність до розладів харчування\n"
                "❌ Трекінг викликає тривогу\n"
                "❌ Відмовляєшся їсти через «ліміт»\n\n"
                "<b>Альтернатива:</b>\n"
                "Метод тарілки + відчуття голоду/ситості\n"
                "без підрахунку — це і є мета NutriSense 💚"
            )
        },
    ]
},
25: {
    "theme": "🏃 Харчування для енергії",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "Чому немає енергії після їжі?",
            "text": (
                "⚡ <b>Тиждень 25 · День 1</b>\n\n"
                "<b>Після їжі хочеться спати? Ось чому.</b>\n\n"
                "Причини втоми після їжі:\n"
                "→ Надто велика порція (переїдання)\n"
                "→ Надлишок простих вуглеводів → стрибок інсуліну\n"
                "→ Дефіцит білку і клітковини\n"
                "→ Мало рідини\n"
                "→ Дефіцит заліза або вітаміну D\n\n"
                "<b>Що дає стабільну енергію:</b>\n"
                "✅ Збалансована тарілка\n"
                "✅ Регулярні прийоми кожні 3-4 год\n"
                "✅ Достатньо білку\n"
                "✅ Складні вуглеводи (не цукор)\n"
                "✅ 1.5-2 л води на день 🌿"
            )
        },
    ]
},
26: {
    "theme": "🌟 Результати та підтримка ваги",
    "posts": [
        {
            "day": 1, "rubric": "поведінка",
            "title": "Як не зірватися після досягнення цілі",
```

```
            "text": (
                "🌟 <b>Тиждень 26 · День 1</b>\n\n"
                "<b>Досягти — легше, ніж утримати. Чому?</b>\n\n"
                "Після досягнення мети:\n"
                "→ Мотивація знижується\n"
                "→ Старі звички «тягнуть» назад\n"
                "→ «Можна собі дозволити» — і поступово все повертається\n\n"
                "<b>Як підтримати результат:</b>\n"
                "✅ Нова ціль (підтримка → якість харчування)\n"
                "✅ Система звичок, а не сила волі\n"
                "✅ Регулярна перевірка — щоміс оцінюй\n"
                "✅ Повернення до базового протоколу при відхиленнях\n"
                "✅ Продовжуй вчитися — NutriSense з тобою 💚"
            )
        },
    ]
},
27: {
    "theme": "🎁 Харчування на святах",
    "posts": [
        {
            "day": 1, "rubric": "поведінка",
            "title": "Як пережити свята без зривів",
            "text": (
                "🎉 <b>Тиждень 27 · День 1</b>\n\n"
                "<b>Свята — не привід для «все або нічого».</b>\n\n"
                "Стратегія NutriSense на свята:\n\n"
                "✅ Не голодуй до святкового столу\n"
                "(з'їж невеликий перекус)\n\n"
                "✅ Починай з овочів і салатів\n\n"
                "✅ Їж повільно і з задоволенням\n\n"
                "✅ Дозволяй собі улюблені страви\n"
                "(але уважно відстежуй насичення)\n\n"
                "✅ Не компенсуй наступного дня голодуванням\n\n"
                "✅ Повернись до звичного режиму\n"
                "наступного ранку — і все 🌿"
            )
        },
    ]
},
28: {
    "theme": "🏆 Підсумок: Твій новий стиль харчування",
    "posts": [
        {
            "day": 1, "rubric": "знання",
            "title": "28 тижнів: що змінилось",
            "text": (
```

```
                    "🏆 <b>Тиждень 28 · Фінал</b>\n\n"
                    "<b>Ти пройшов(ла) весь курс NutriSense!</b>\n\n"
                    "За ці 28 тижнів ти:\n"
                    "✅ Навчився розрізняти голод\n"
                    "✅ Зрозумів/зрозуміла роль кожного макронутрієнту\n"
                    "✅ Склав/склала свій баланс тарілки\n"
                    "✅ Позбувся/позбулась заборон і страху перед їжею\n"
                    "✅ Дізнався/дізналась як сон, стрес і гормони впливають\n"
                    "на харчування\n"
                    "✅ Отримав інструменти на все життя\n\n"
                    "<b>Харчування — це не дієта.\n"
                    "Це твій спосіб жити.</b>\n\n"
                    "NutriSense завжди поруч 🌿💚"
                )
            },
        ]
    },
}


# ────────────────────────────────────────────────
# СТАНИ (FSM)
# ────────────────────────────────────────────────
class KBJUState(StatesGroup):
    waiting_gender   = State()
    waiting_age      = State()
    waiting_weight   = State()
    waiting_height   = State()
    waiting_activity = State()
    waiting_goal     = State()

class BehaviorTestState(StatesGroup):
    waiting_answer = State()

class AdminBroadcastState(StatesGroup):
    waiting_text = State()

class AdminPaymentState(StatesGroup):
    waiting_user_id = State()
    waiting_plan    = State()


# ────────────────────────────────────────────────
# DATABASE
# ────────────────────────────────────────────────
async def init_db():
    async with aiosqlite.connect(DB_PATH) as db:
```

```
    await db.executescript("""
        CREATE TABLE IF NOT EXISTS users (
            user_id     INTEGER PRIMARY KEY,
            username    TEXT,
            full_name   TEXT,
            plan        TEXT DEFAULT 'free',
            plan_until  TEXT,
            joined_at   TEXT DEFAULT CURRENT_TIMESTAMP,
            week        INTEGER DEFAULT 1,
            last_post   TEXT,
            gender      TEXT,
            age         INTEGER,
            weight      REAL,
            height      REAL,
            activity    REAL,
            goal        TEXT,
            calories    INTEGER,
            protein     REAL,
            fat         REAL,
            carbs       REAL
        );

        CREATE TABLE IF NOT EXISTS payments (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id     INTEGER,
            plan        TEXT,
            amount_uah  INTEGER,
            invoice_id  TEXT,
            status      TEXT DEFAULT 'pending',
            created_at  TEXT DEFAULT CURRENT_TIMESTAMP,
            paid_at     TEXT
        );

        CREATE TABLE IF NOT EXISTS content_log (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id     INTEGER,
            week        INTEGER,
            day         INTEGER,
            rubric      TEXT,
            sent_at     TEXT DEFAULT CURRENT_TIMESTAMP
        );

        CREATE TABLE IF NOT EXISTS tracker (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id     INTEGER,
            date        TEXT,
            meal        TEXT,
```

```
                note        TEXT,
                created_at  TEXT DEFAULT CURRENT_TIMESTAMP
            );
        """)
        await db.commit()
    log.info("✅ База даних ініціалізована")


async def get_user(user_id: int) -> dict | None:
    async with aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row
        async with db.execute(
            "SELECT * FROM users WHERE user_id = ?", (user_id,)
        ) as cur:
            row = await cur.fetchone()
            return dict(row) if row else None


async def upsert_user(user_id: int, **kwargs):
    async with aiosqlite.connect(DB_PATH) as db:
        existing = await get_user(user_id)
        if not existing:
            await db.execute(
                "INSERT INTO users (user_id) VALUES (?)", (user_id,)
            )
        if kwargs:
            sets = ", ".join(f"{k} = ?" for k in kwargs)
            vals = list(kwargs.values()) + [user_id]
            await db.execute(f"UPDATE users SET {sets} WHERE user_id = ?", vals)
        await db.commit()


async def get_all_users() -> list[dict]:
    async with aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row
        async with db.execute("SELECT * FROM users") as cur:
            rows = await cur.fetchall()
            return [dict(r) for r in rows]


async def get_stats() -> dict:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT COUNT(*) FROM users") as c:
            total = (await c.fetchone())[0]
        async with db.execute(
            "SELECT COUNT(*) FROM users WHERE plan != 'free'"
        ) as c:
```

```
            paid = (await c.fetchone())[0]
        async with db.execute(
            "SELECT COUNT(*) FROM payments WHERE status = 'paid'"
        ) as c:
            payments = (await c.fetchone())[0]
        async with db.execute(
            "SELECT SUM(amount_uah) FROM payments WHERE status = 'paid'"
        ) as c:
            revenue = (await c.fetchone())[0] or 0
    return {
        "total": total, "paid": paid,
        "payments": payments, "revenue": revenue
    }


async def save_payment(user_id: int, plan: str, amount: int, invoice_id: str):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            """INSERT INTO payments (user_id, plan, amount_uah, invoice_id, statu
               VALUES (?, ?, ?, ?, 'pending')""",
            (user_id, plan, amount, invoice_id)
        )
        await db.commit()


async def confirm_payment(invoice_id: str):
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT user_id, plan FROM payments WHERE invoice_id = ?",
            (invoice_id,)
        ) as cur:
            row = await cur.fetchone()
        if not row:
            return None
        user_id, plan = row
        plan_until = (
            datetime.now() + timedelta(days=PLANS[plan]["days"])
        ).strftime("%Y-%m-%d %H:%M:%S")
        await db.execute(
            """UPDATE payments SET status = 'paid', paid_at = CURRENT_TIMESTAMP
               WHERE invoice_id = ?""", (invoice_id,)
        )
        await db.execute(
            "UPDATE users SET plan = ?, plan_until = ? WHERE user_id = ?",
            (plan, plan_until, user_id)
        )
        await db.commit()
```

```
        return user_id, plan


def has_access(user: dict, required: str) -> bool:
    """Перевірка доступу до плану"""
    order = {"free": 0, "start": 1, "premium": 2, "vip": 3}
    if not user:
        return False
    plan = user.get("plan", "free")
    until_str = user.get("plan_until")
    if plan != "free" and until_str:
        until = datetime.strptime(until_str, "%Y-%m-%d %H:%M:%S")
        if datetime.now() > until:
            return False
    return order.get(plan, 0) >= order.get(required, 0)


# ────────────────────────────────────────────────
# KBJU CALCULATOR (Формула Міффліна-Сан Жеора)
# ────────────────────────────────────────────────
ACTIVITY_LEVELS = {
    "sedentary":   (1.2,  "Малорухливий (офіс, без спорту)"),
    "light":       (1.375,"Легка активність (1-3 тренування/тиж)"),
    "moderate":    (1.55, "Помірна активність (3-5 тренувань/тиж)"),
    "active":      (1.725,"Висока активність (6-7 тренувань/тиж)"),
    "very_active": (1.9,  "Дуже висока (спортсмен/фізична робота)"),
}

GOALS = {
    "lose":     ("Схуднення (-300 ккал)",      -300),
    "maintain": ("Підтримка ваги (0 ккал)",       0),
    "gain":     ("Набір маси (+300 ккал)",      +300),
}


def calculate_kbju(gender: str, age: int, weight: float,
                   height: float, activity: float, goal: str) -> dict:
    """Розрахунок КБЖУ за Міффліним-Сан Жеором"""
    if gender == "female":
        bmr = 10 * weight + 6.25 * height - 5 * age - 161
    else:
        bmr = 10 * weight + 6.25 * height - 5 * age + 5

    tdee = bmr * activity
    goal_delta = GOALS[goal][1]
    calories = round(tdee + goal_delta)
```

```
    # Розподіл макронутрієнтів
    protein = round(weight * 1.7)            # 1.7 г/кг
    fat     = round(calories * 0.28 / 9)     # 28% калорій з жирів
    carbs   = round((calories - protein*4 - fat*9) / 4)

    return {
        "bmr":     round(bmr),
        "tdee":    round(tdee),
        "calories": calories,
        "protein":  protein,
        "fat":      fat,
        "carbs":    carbs,
    }


def build_meal_plan(calories: int, protein: float,
                    fat: float, carbs: float) -> str:
    """Генерація плану харчування по прийомах"""
    # Розподіл по прийомах (%)
    dist = {
        "🌅 Сніданок":  (0.30, 0.25, 0.35, 0.25),
        "🍱 Перекус 1": (0.10, 0.15, 0.10, 0.10),
        "☀️ Обід":      (0.35, 0.35, 0.30, 0.40),
        "🍎 Перекус 2": (0.10, 0.15, 0.10, 0.10),
        "🌙 Вечеря":    (0.15, 0.10, 0.15, 0.15),
    }
    lines = ["<b>📋 Розподіл по прийомах їжі:</b>\n"]
    for meal, (kc, pr, ft, cb) in dist.items():
        lines.append(
            f"{meal}\n"
            f"  <code>Калорії: {round(calories*kc)} ккал\n"
            f"  Білок:   {round(protein*pr)} г\n"
            f"  Жири:    {round(fat*ft)} г\n"
            f"  Вуглев.: {round(carbs*cb)} г</code>\n"
        )
    return "\n".join(lines)


# ────────────────────────────────────────────────
# MONOBANK ACQUIRING
# ────────────────────────────────────────────────
MONO_API_URL = "https://api.monobank.ua/api/merchant/invoice/create"

async def create_mono_invoice(
    user_id: int, plan: str, amount_kopiy: int, redirect_url: str = ""
) -> str | None:
    """Створення інвойсу через Monobank Acquiring"""
```

```
    payload = {
        "amount":      amount_kopiy,
        "ccy":         980,           # UAH
        "merchantPaymInfo": {
            "reference":   f"nutrisense_{user_id}_{plan}_{int(datetime.now().time
            "destination": f"NutriSense {PLANS[plan]['name']} — підписка 30 днів"
            "basketOrder": [{
                "name":  f"NutriSense {PLANS[plan]['name']}",
                "qty":   1,
                "sum":   amount_kopiy,
                "icon":  "",
                "unit":  "шт.",
                "code":  plan,
            }]
        },
        "redirectUrl": redirect_url or f"https://t.me/{CHANNEL_ID.replace('@','')
        "webHookUrl":  f"{WEBHOOK_URL}/mono_webhook" if WEBHOOK_URL else "",
        "saveCardData": {
            "saveCard":    True,        # Зберігаємо картку для повторних платежі
            "walletId":    str(user_id)
        }
    }
    headers = {"X-Token": MONO_TOKEN, "Content-Type": "application/json"}
    try:
        async with aiohttp.ClientSession() as session:
            async with session.post(
                MONO_API_URL, json=payload, headers=headers
            ) as resp:
                if resp.status == 200:
                    data = await resp.json()
                    invoice_id = data.get("invoiceId", "")
                    page_url   = data.get("pageUrl", "")
                    await save_payment(user_id, plan, amount_kopiy // 100, invoic
                    return page_url
                else:
                    err = await resp.text()
                    log.error(f"Monobank error {resp.status}: {err}")
                    return None
    except Exception as e:
        log.error(f"Monobank request failed: {e}")
        return None


async def check_mono_invoice(invoice_id: str) -> str:
    """Перевірка статусу інвойсу"""
    url = f"https://api.monobank.ua/api/merchant/invoice/status?invoiceId={invoic
    headers = {"X-Token": MONO_TOKEN}
```

```
    try:
        async with aiohttp.ClientSession() as session:
            async with session.get(url, headers=headers) as resp:
                if resp.status == 200:
                    data = await resp.json()
                    return data.get("status", "unknown")
    except Exception as e:
        log.error(f"Mono status check failed: {e}")
    return "error"


# ────────────────────────────────────────────
# КЛАВІАТУРИ
# ────────────────────────────────────────────
def kb_main_menu(plan: str = "free") -> InlineKeyboardMarkup:
    b = InlineKeyboardBuilder()
    b.button(text="📊 Розрахувати КБЖУ",     callback_data="kbju_start")
    b.button(text="🍽️ Мій план харчування",   callback_data="my_plan")
    b.button(text="🧠 Тест поведінки",        callback_data="behavior_test")
    b.button(text="📚 Контент тижня",         callback_data="weekly_content")
    if plan in ("premium", "vip"):
        b.button(text="📸 Аналіз тарілки",    callback_data="plate_analysis")
    if plan == "vip":
        b.button(text="🤖 ШІ-нутриціолог",    callback_data="ai_nutritionist")
    b.button(text="📅 Трекер їжі",            callback_data="tracker")
    b.button(text="💎 Тарифи",                callback_data="plans")
    b.button(text="👤 Мій профіль",           callback_data="my_profile")
    b.adjust(2, 2, 2, 2, 1)
    return b.as_markup()


def kb_plans() -> InlineKeyboardMarkup:
    b = InlineKeyboardBuilder()
    b.button(text="🌱 Start — 199 ₴/міс",    callback_data="buy_start")
    b.button(text="🌿 Premium — 349 ₴/міс",  callback_data="buy_premium")
    b.button(text="👑 VIP — 549 ₴/міс",      callback_data="buy_vip")
    b.button(text="⬅️ Назад",                 callback_data="main_menu")
    b.adjust(1)
    return b.as_markup()


def kb_confirm_buy(plan: str) -> InlineKeyboardMarkup:
    b = InlineKeyboardBuilder()
    b.button(text=f"💳 Оплатити карткою",   callback_data=f"pay_card_{plan}")
    b.button(text="⬅️ Назад до тарифів",    callback_data="plans")
    b.adjust(1)
    return b.as_markup()
```

```python
def kb_activity() -> InlineKeyboardMarkup:
    b = InlineKeyboardBuilder()
    for key, (_, label) in ACTIVITY_LEVELS.items():
        b.button(text=label, callback_data=f"activity_{key}")
    b.adjust(1)
    return b.as_markup()


def kb_goal() -> InlineKeyboardMarkup:
    b = InlineKeyboardBuilder()
    for key, (label, _) in GOALS.items():
        b.button(text=label, callback_data=f"goal_{key}")
    b.adjust(1)
    return b.as_markup()


def kb_gender() -> InlineKeyboardMarkup:
    b = InlineKeyboardBuilder()
    b.button(text="👩 Жінка", callback_data="gender_female")
    b.button(text="👨 Чоловік", callback_data="gender_male")
    b.adjust(2)
    return b.as_markup()


def kb_back_main() -> InlineKeyboardMarkup:
    b = InlineKeyboardBuilder()
    b.button(text="🏠 Головне меню", callback_data="main_menu")
    b.adjust(1)
    return b.as_markup()


def kb_admin() -> InlineKeyboardMarkup:
    b = InlineKeyboardBuilder()
    b.button(text="📊 Статистика",          callback_data="adm_stats")
    b.button(text="👥 Всі юзери",           callback_data="adm_users")
    b.button(text="📢 Розсилка",            callback_data="adm_broadcast")
    b.button(text="✅ Підтвердити оплату",  callback_data="adm_confirm_pay")
    b.button(text="📤 Контент тижня всім",  callback_data="adm_send_content")
    b.adjust(2, 2, 1)
    return b.as_markup()


def kb_behavior_test(q_num: int) -> InlineKeyboardMarkup:
    """Клавіатура для тесту харчової поведінки"""
    options = {
```

```
        0: [("👍 Так, часто",   "bt_yes"),
            ("🤔 Іноді",        "bt_sometimes"),
            ("👎 Рідко/ніколи", "bt_no")],
        1: [("🟢 Наростає поступово", "bt_gradual"),
            ("🔴 Виникає раптово",    "bt_sudden"),
            ("⚪ По-різному",         "bt_varies")],
        2: [("😔 Від стресу/тривоги",  "bt_stress"),
            ("😴 Від нудьги",           "bt_bored"),
            ("😋 Від запаху/вигляду",  "bt_sensory")],
        3: [("✅ Так, легко",           "bt_easy"),
            ("⚠️ Важко, але можу",    "bt_hard"),
            ("❌ Не можу зупинитись", "bt_cant")],
        4: [("😔 Часто соромлюсь",  "bt_shame"),
            ("😔 Іноді",             "bt_sometimes2"),
            ("😊 Рідко/ніколи",     "bt_never")],
    }
    b = InlineKeyboardBuilder()
    for text, cb in options.get(q_num, []):
        b.button(text=text, callback_data=cb)
    b.adjust(1)
    return b.as_markup()


# ────────────────────────────────────────────
# ТЕСТ ХАРЧОВОЇ ПОВЕДІНКИ
# ────────────────────────────────────────────
BEHAVIOR_QUESTIONS = [
    "1️⃣ Чи буває що їси без фізичного голоду\n(від нудьги, стресу, емоцій)?",
    "2️⃣ Як зазвичай з'являється голод?",
    "3️⃣ Що найчастіше провокує їсти\nне в «свій час»?",
    "4️⃣ Чи вдається зупинитись\nпісля першого шматочка улюбленого?",
    "5️⃣ Чи виникає сором або провина\nпісля певних прийомів їжі?",
]


def analyze_behavior(answers: list[str]) -> str:
    """Аналіз відповідей тесту"""
    score = 0
    if "bt_yes" in answers:      score += 2
    if "bt_sometimes" in answers: score += 1
    if "bt_sudden" in answers:   score += 2
    if "bt_stress" in answers:   score += 2
    if "bt_bored" in answers:    score += 1
    if "bt_cant" in answers:     score += 3
    if "bt_shame" in answers:    score += 2

    if score <= 3:
```

```
        return (
            "🟢 <b>Гомеостатичний тип харчової поведінки</b>\n\n"
            "Харчування відбувається переважно за фізичним голодом.\n"
            "Насичення відчувається добре, їжа без причини — рідкість.\n\n"
            "Рекомендації:\n"
            "✅ Продовжуй слухати своє тіло\n"
            "✅ Стеж за балансом макронутрієнтів\n"
            "✅ Перевір розрахунок КБЖУ /kbju"
        )
    elif score <= 7:
        return (
            "🟡 <b>Гедонічний тип харчової поведінки</b>\n\n"
            "Іноді трапляється їсти заради задоволення або від емоцій.\n"
            "Це нормально! Але є простір для роботи.\n\n"
            "Рекомендації:\n"
            "🔶 Визначай рівень голоду перед їжею (шкала 1-10)\n"
            "🔶 Знайди альтернативи для емоційного заспокоєння\n"
            "🔶 Не забороняй улюблене — дозволяй усвідомлено\n"
            "🔶 Регулярні прийоми їжі зменшать тягу"
        )
    else:
        return (
            "🔴 <b>Дисрегульований тип харчової поведінки</b>\n\n"
            "Харчування часто некероване або з сильними емоціями.\n"
            "Це сигнал: тілу і психіці потрібна підтримка.\n\n"
            "Рекомендації:\n"
            "🔴 Регулярні прийоми їжі кожні 3-4 год\n"
            "🔴 Достатня калорійність (не менше 1400 ккал)\n"
            "🔴 Якісний сон 7-9 годин\n"
            "🔴 Зниження рівня стресу\n"
            "🔴 Розгляни роботу з психологом або нутриціологом"
        )


# ────────────────────────────────────────────
# РОУТЕРИ
# ────────────────────────────────────────────
router = Router()


# ── /start ──────────────────────────────
@router.message(CommandStart())
async def cmd_start(msg: Message):
    user = await get_user(msg.from_user.id)
    await upsert_user(
        msg.from_user.id,
        username=msg.from_user.username or "",
```

```
        full_name=msg.from_user.full_name or "",
    )
    user = await get_user(msg.from_user.id)
    plan = user.get("plan", "free") if user else "free"

    text = (
        "🌿 <b>Вітаю в NutriSense!</b>\n\n"
        "Я твій особистий нутриціолог у Telegram.\n\n"
        "Що я вмію:\n"
        "📊 Точно розраховую КБЖУ за Міффліном\n"
        "🍽️ Складаю план харчування по прийомах\n"
        "🧠 Визначаю тип харчової поведінки\n"
        "📚 Надсилаю контент 28 тижнів про харчування\n"
        "📸 Аналізую фото твоєї тарілки (Premium)\n"
        "🤖 Відповідаю на будь-які питання (VIP)\n\n"
        "<b>Перший розрахунок КБЖУ — безкоштовно!</b>\n"
        "Обирай що хочеш зробити 👇"
    )
    await msg.answer(text, reply_markup=kb_main_menu(plan))


# ── /menu ──────────────────────────────────
@router.message(Command("menu"))
async def cmd_menu(msg: Message):
    user = await get_user(msg.from_user.id)
    plan = user.get("plan", "free") if user else "free"
    await msg.answer("🏠 Головне меню:", reply_markup=kb_main_menu(plan))


# ── /kbju ──────────────────────────────────
@router.message(Command("kbju"))
async def cmd_kbju(msg: Message, state: FSMContext):
    await state.set_state(KBJUState.waiting_gender)
    await msg.answer(
        "📊 <b>Розрахунок КБЖУ</b>\n\n"
        "Визначимо точну норму калорій, білків, жирів і вуглеводів.\n\n"
        "👇 Обери стать:",
        reply_markup=kb_gender()
    )


@router.callback_query(F.data.startswith("gender_"), KBJUState.waiting_gender)
async def kbju_gender(cq: CallbackQuery, state: FSMContext):
    gender = cq.data.split("_")[1]
    await state.update_data(gender=gender)
    await state.set_state(KBJUState.waiting_age)
    await cq.message.edit_text(
```

```
        "📊 <b>Розрахунок КБЖУ</b>\n\n"
        "Введи свій <b>вік</b> (числом, наприклад: 27):"
    )
    await cq.answer()


@router.message(KBJUState.waiting_age)
async def kbju_age(msg: Message, state: FSMContext):
    try:
        age = int(msg.text.strip())
        if not 10 <= age <= 100:
            raise ValueError
        await state.update_data(age=age)
        await state.set_state(KBJUState.waiting_weight)
        await msg.answer("Введи свою <b>вагу</b> (кг, наприклад: 65):")
    except ValueError:
        await msg.answer("⚠️ Введи коректний вік (10–100):")


@router.message(KBJUState.waiting_weight)
async def kbju_weight(msg: Message, state: FSMContext):
    try:
        weight = float(msg.text.strip().replace(",", "."))
        if not 30 <= weight <= 300:
            raise ValueError
        await state.update_data(weight=weight)
        await state.set_state(KBJUState.waiting_height)
        await msg.answer("Введи свій <b>зріст</b> (см, наприклад: 168):")
    except ValueError:
        await msg.answer("⚠️ Введи коректну вагу (30–300 кг):")


@router.message(KBJUState.waiting_height)
async def kbju_height(msg: Message, state: FSMContext):
    try:
        height = float(msg.text.strip().replace(",", "."))
        if not 100 <= height <= 250:
            raise ValueError
        await state.update_data(height=height)
        await state.set_state(KBJUState.waiting_activity)
        await msg.answer(
            "Обери рівень <b>фізичної активності</b>:",
            reply_markup=kb_activity()
        )
    except ValueError:
        await msg.answer("⚠️ Введи коректний зріст (100–250 см):")
```

```
@router.callback_query(F.data.startswith("activity_"), KBJUState.waiting_activity
async def kbju_activity(cq: CallbackQuery, state: FSMContext):
    key = cq.data.replace("activity_", "")
    coef, label = ACTIVITY_LEVELS[key]
    await state.update_data(activity=coef, activity_label=label)
    await state.set_state(KBJUState.waiting_goal)
    await cq.message.edit_text(
        "Яка твоя <b>ціль</b>?",
        reply_markup=kb_goal()
    )
    await cq.answer()


@router.callback_query(F.data.startswith("goal_"), KBJUState.waiting_goal)
async def kbju_goal(cq: CallbackQuery, state: FSMContext):
    goal = cq.data.replace("goal_", "")
    data = await state.get_data()
    await state.clear()

    result = calculate_kbju(
        gender=data["gender"],
        age=data["age"],
        weight=data["weight"],
        height=data["height"],
        activity=data["activity"],
        goal=goal
    )

    await upsert_user(
        cq.from_user.id,
        gender=data["gender"],
        age=data["age"],
        weight=data["weight"],
        height=data["height"],
        activity=data["activity"],
        goal=goal,
        calories=result["calories"],
        protein=result["protein"],
        fat=result["fat"],
        carbs=result["carbs"],
    )

    meal_plan = build_meal_plan(
        result["calories"], result["protein"],
        result["fat"], result["carbs"]
    )
```

```
    goal_label = GOALS[goal][0]
    gender_label = "Жінка" if data["gender"] == "female" else "Чоловік"

    text = (
        f"✅ <b>Твій розрахунок КБЖУ</b>\n\n"
        f"👤 {gender_label}, {data['age']} р., "
        f"{data['weight']} кг, {data['height']} см\n"
        f"🏃 {data['activity_label']}\n"
        f"🎯 {goal_label}\n\n"
        f"━━━━━━━━━━━━━━━━━━━━\n"
        f"🔥 <b>Базовий метаболізм (BMR):</b> {result['bmr']} ккал\n"
        f"⚡ <b>Норма калорій (TDEE):</b> {result['tdee']} ккал\n"
        f"🎯 <b>З урахуванням цілі:</b> {result['calories']} ккал\n"
        f"━━━━━━━━━━━━━━━━━━━━\n"
        f"🥩 <b>Білок:</b>    {result['protein']} г\n"
        f"🧈 <b>Жири:</b>     {result['fat']} г\n"
        f"🌾 <b>Вуглеводи:</b> {result['carbs']} г\n"
        f"━━━━━━━━━━━━━━━━━━━━\n\n"
        f"{meal_plan}"
    )

    user = await get_user(cq.from_user.id)
    plan = user.get("plan", "free") if user else "free"

    await cq.message.edit_text(text, reply_markup=kb_main_menu(plan))
    await cq.answer("✅ Розрахунок готовий!")


# ── CALLBACK: main_menu ──────────────────────
@router.callback_query(F.data == "main_menu")
async def cb_main_menu(cq: CallbackQuery):
    user = await get_user(cq.from_user.id)
    plan = user.get("plan", "free") if user else "free"
    await cq.message.edit_text(
        "🏠 <b>Головне меню NutriSense</b>",
        reply_markup=kb_main_menu(plan)
    )
    await cq.answer()


# ── CALLBACK: kbju_start ─────────────────────
@router.callback_query(F.data == "kbju_start")
async def cb_kbju_start(cq: CallbackQuery, state: FSMContext):
    await state.set_state(KBJUState.waiting_gender)
    await cq.message.edit_text(
        "📊 <b>Розрахунок КБЖУ</b>\n\n"
```

```
        "Обери стать:",
        reply_markup=kb_gender()
    )
    await cq.answer()


# ── CALLBACK: my_plan ──────────────────────
@router.callback_query(F.data == "my_plan")
async def cb_my_plan(cq: CallbackQuery):
    user = await get_user(cq.from_user.id)
    if not user or not user.get("calories"):
        await cq.answer("⚠️ Спочатку розрахуй КБЖУ!", show_alert=True)
        return

    meal = build_meal_plan(
        user["calories"], user["protein"], user["fat"], user["carbs"]
    )
    goal_label = GOALS.get(user["goal"], ("Не вказано",))[0]

    text = (
        f"🍽️ <b>Твій план харчування</b>\n\n"
        f"🎯 Ціль: {goal_label}\n"
        f"🔥 Норма: {user['calories']} ккал/день\n"
        f"🥩 Б: {user['protein']}г  🧈 Ж: {user['fat']}г  "
        f"🌾 В: {user['carbs']}г\n\n"
        f"{meal}\n"
        f"━━━━━━━━━━━━━━━━━━━━\n"
        f"<b>📌 Рекомендовані продукти:</b>\n\n"
        f"🌾 <b>Крупи:</b> гречка, овес, рис, булгур, кінва\n"
        f"🥩 <b>Білок:</b> курка, індичка, тріска, яйця, нут\n"
        f"🥑 <b>Жири:</b> авокадо, оливкова олія, горіхи\n"
        f"🥦 <b>Овочі:</b> броколі, шпинат, кабачок, перець\n"
        f"🍎 <b>Фрукти:</b> ягоди, ківі, яблуко\n"
        f"🦠 <b>Пробіотики:</b> йогурт, кефір, квашена капуста"
    )
    await cq.message.edit_text(text, reply_markup=kb_back_main())
    await cq.answer()


# ── CALLBACK: weekly_content ─────────────────
@router.callback_query(F.data == "weekly_content")
async def cb_weekly_content(cq: CallbackQuery):
    user = await get_user(cq.from_user.id)
    if not has_access(user, "start"):
        await cq.answer(
            "🔒 Контент доступний з тарифу Start",
            show_alert=True
```

```
        )
        return

    week_num = user.get("week", 1) or 1
    week_data = WEEKLY_CONTENT.get(week_num, WEEKLY_CONTENT[1])
    posts = week_data["posts"]

    text = (
        f"📚 <b>Тиждень {week_num}: {week_data['theme']}</b>\n\n"
        f"На цьому тижні {len(posts)} матеріали:\n\n"
    )
    for p in posts:
        text += f"📌 День {p['day']}: {p['title']}\n"

    text += f"\n<i>Загальний прогрес: тиждень {week_num} з 28</i>"

    b = InlineKeyboardBuilder()
    for i, p in enumerate(posts):
        b.button(
            text=f"📖 День {p['day']}: {p['title'][:25]}",
            callback_data=f"read_post_{week_num}_{i}"
        )
    b.button(text="◀️ Назад", callback_data="main_menu")
    b.adjust(1)

    await cq.message.edit_text(text, reply_markup=b.as_markup())
    await cq.answer()


@router.callback_query(F.data.startswith("read_post_"))
async def cb_read_post(cq: CallbackQuery):
    parts = cq.data.split("_")
    week_num, post_idx = int(parts[2]), int(parts[3])
    week_data = WEEKLY_CONTENT.get(week_num)
    if not week_data or post_idx >= len(week_data["posts"]):
        await cq.answer("⚠️ Матеріал не знайдено", show_alert=True)
        return

    post = week_data["posts"][post_idx]
    b = InlineKeyboardBuilder()
    if post_idx > 0:
        b.button(text="◀️ Попередній", callback_data=f"read_post_{week_num}_{post
    if post_idx < len(week_data["posts"]) - 1:
        b.button(text="Наступний ▶️", callback_data=f"read_post_{week_num}_{post_
    b.button(text="📚 До тижня", callback_data="weekly_content")
    b.adjust(2, 1)
```

```
    await cq.message.edit_text(post["text"], reply_markup=b.as_markup())
    await cq.answer()


# ── CALLBACK: behavior_test ──────────────────
@router.callback_query(F.data == "behavior_test")
async def cb_behavior_test(cq: CallbackQuery, state: FSMContext):
    await state.set_state(BehaviorTestState.waiting_answer)
    await state.update_data(answers=[], q_num=0)
    await cq.message.edit_text(
        "🧠 <b>Тест харчової поведінки</b>\n\n"
        "5 питань визначать твій тип харчової поведінки\n"
        "та дадуть персональні рекомендації.\n\n"
        f"{BEHAVIOR_QUESTIONS[0]}",
        reply_markup=kb_behavior_test(0)
    )
    await cq.answer()


@router.callback_query(
    BehaviorTestState.waiting_answer,
    F.data.startswith("bt_")
)
async def cb_behavior_answer(cq: CallbackQuery, state: FSMContext):
    data = await state.get_data()
    answers = data.get("answers", [])
    q_num   = data.get("q_num", 0)

    answers.append(cq.data)
    q_num += 1

    if q_num >= len(BEHAVIOR_QUESTIONS):
        await state.clear()
        result_text = analyze_behavior(answers)
        b = InlineKeyboardBuilder()
        b.button(text="🏠 Головне меню", callback_data="main_menu")
        await cq.message.edit_text(
            f"🧠 <b>Результат тесту</b>\n\n{result_text}",
            reply_markup=b.as_markup()
        )
    else:
        await state.update_data(answers=answers, q_num=q_num)
        await cq.message.edit_text(
            f"<b>Питання {q_num+1}/5</b>\n\n{BEHAVIOR_QUESTIONS[q_num]}",
            reply_markup=kb_behavior_test(q_num)
        )
    await cq.answer()
```

```
# ── CALLBACK: plans ──────────────────────────
@router.callback_query(F.data == "plans")
async def cb_plans(cq: CallbackQuery):
    text = (
        "💎 <b>Тарифи NutriSense</b>\n\n"
        "Підписка підключається одразу після оплати карткою.\n\n"
    )
    for key, plan in PLANS.items():
        text += (
            f"{plan['emoji']} <b>{plan['name']}</b> — "
            f"{plan['price_uah']} ₴/міс\n"
        )
        for f in plan["features"]:
            text += f"  {f}\n"
        text += "\n"

    await cq.message.edit_text(text, reply_markup=kb_plans())
    await cq.answer()


# ── BUY CALLBACKS ────────────────────────────
@router.callback_query(F.data.startswith("buy_"))
async def cb_buy(cq: CallbackQuery):
    plan = cq.data.replace("buy_", "")
    if plan not in PLANS:
        await cq.answer("⚠️ Тариф не знайдено", show_alert=True)
        return

    p = PLANS[plan]
    text = (
        f"{p['emoji']} <b>{p['name']}</b>\n\n"
        f"💰 Вартість: <b>{p['price_uah']} ₴/місяць</b>\n"
        f"📅 Доступ: 30 днів\n\n"
        f"<b>Включає:</b>\n"
        + "\n".join(p["features"])
        + "\n\n"
        f"Оплата через Monobank — Visa/Mastercard 💳"
    )
    await cq.message.edit_text(text, reply_markup=kb_confirm_buy(plan))
    await cq.answer()


@router.callback_query(F.data.startswith("pay_card_"))
async def cb_pay_card(cq: CallbackQuery):
    plan = cq.data.replace("pay_card_", "")
```

```
if plan not in PLANS:
    await cq.answer("⚠️ Помилка", show_alert=True)
    return

p = PLANS[plan]
user = cq.from_user

# Зберігаємо очікуваний платіж в БД
await save_payment(
    user_id=user.id,
    plan=plan,
    amount=p["price_uah"],
    invoice_id=f"jar_{user.id}_{plan}_{int(datetime.now().timestamp())}"
)

# Повідомлення адміну про новий запит на оплату
try:
    await cq.bot.send_message(
        ADMIN_ID,
        f"💰 <b>Новий запит на оплату!</b>\n\n"
        f"👤 Юзер: {user.full_name} (@{user.username or '—'})\n"
        f"🆔 ID: <code>{user.id}</code>\n"
        f"💎 Тариф: {p['name']}\n"
        f"💵 Сума: {p['price_uah']} ₴\n\n"
        f"Після отримання оплати активуй підписку:\n"
        f"/admin → ✅ Підтвердити оплату → {user.id}"
    )
except Exception as e:
    log.warning(f"Cannot notify admin: {e}")

b = InlineKeyboardBuilder()
b.button(text="💳 Перейти до оплати", url=MONO_JAR_URL)
b.button(text="✅ Я оплатив(ла)", callback_data=f"paid_notify_{plan}")
b.button(text="◀️ Назад", callback_data="plans")
b.adjust(1)

await cq.message.edit_text(
    f"💳 <b>Оплата {p['name']}</b>\n\n"
    f"💵 Сума: <b>{p['price_uah']} ₴</b>\n\n"
    f"1️⃣ Натисни <b>«Перейти до оплати»</b>\n"
    f"2️⃣ Переведи <b>рівно {p['price_uah']} ₴</b>\n"
    f"3️⃣ Натисни <b>«Я оплатив(ла)»</b> — ми отримаємо сповіщення\n"
    f"4️⃣ Підписка активується протягом кількох хвилин ⚡\n\n"
    f"<i>Питання? Напиши адміністратору</i>",
    reply_markup=b.as_markup()
)
await cq.answer()
```

```
@router.callback_query(F.data.startswith("paid_notify_"))
async def cb_paid_notify(cq: CallbackQuery):
    """Клієнт натиснув 'Я оплатив' — сповіщаємо адміна з кнопкою активації"""
    plan = cq.data.replace("paid_notify_", "")
    user = cq.from_user
    p = PLANS.get(plan, {})

    # Кнопка одним натисканням активує підписку
    b = InlineKeyboardBuilder()
    b.button(
        text=f"✅ Активувати {p.get('name', plan)}",
        callback_data=f"adm_oneclick_{user.id}_{plan}"
    )
    b.button(
        text="❌ Відхилити",
        callback_data=f"adm_reject_{user.id}"
    )
    b.adjust(1)

    try:
        await cq.bot.send_message(
            ADMIN_ID,
            f"🔔 <b>Клієнт каже що оплатив!</b>\n\n"
            f"👤 {user.full_name} (@{user.username or '—'})\n"
            f"💎 Тариф: {p.get('name', plan)}\n"
            f"💵 Сума: {p.get('price_uah', '?')} ₴\n\n"
            f"Перевір банку і натисни кнопку 👇",
            reply_markup=b.as_markup()
        )
    except Exception as e:
        log.warning(f"Cannot notify admin: {e}")

    await cq.message.edit_text(
        "✅ <b>Дякуємо!</b>\n\n"
        "Ми отримали сповіщення і перевіряємо оплату.\n"
        "Підписка буде активована протягом кількох хвилин.\n\n"
        "Якщо виникли питання — напиши адміністратору 🌿",
        reply_markup=kb_back_main()
    )
    await cq.answer("✅ Сповіщення надіслано!")


@router.callback_query(F.data.startswith("adm_oneclick_"))
async def adm_oneclick_activate(cq: CallbackQuery):
    """Адмін натиснув '✅ Активувати' — одне натискання"""
```

```
    if not is_admin(cq.from_user.id):
        return

    parts = cq.data.split("_")
    # adm_oneclick_{user_id}_{plan}
    user_id = int(parts[2])
    plan    = parts[3]

    plan_until = (
        datetime.now() + timedelta(days=PLANS[plan]["days"])
    ).strftime("%Y-%m-%d %H:%M:%S")

    await upsert_user(user_id, plan=plan, plan_until=plan_until)

    # Повідомлення клієнту
    try:
        await cq.bot.send_message(
            user_id,
            f"🎉 <b>Підписку активовано!</b>\n\n"
            f"Тариф: {PLANS[plan]['name']}\n"
            f"Діє до: {plan_until[:10]}\n\n"
            f"Дякуємо за довіру! 🌿\n"
            f"/menu — перейти до меню"
        )
    except Exception as e:
        log.warning(f"Cannot notify user {user_id}: {e}")

    # Оновлюємо повідомлення адміну
    await cq.message.edit_text(
        f"✅ <b>Активовано!</b>\n\n"
        f"Юзер {user_id} — {PLANS[plan]['name']}\n"
        f"До: {plan_until[:10]}"
    )
    await cq.answer("✅ Підписку активовано!")


@router.callback_query(F.data.startswith("adm_reject_"))
async def adm_reject_payment(cq: CallbackQuery):
    """Адмін відхилив оплату"""
    if not is_admin(cq.from_user.id):
        return

    user_id = int(cq.data.split("_")[2])

    try:
        await cq.bot.send_message(
            user_id,
```

```
            "⚠️ <b>Оплату не знайдено.</b>\n\n"
            "Будь ласка, перевір чи правильно переведена сума "
            "і спробуй знову або напиши адміністратору."
        )
    except Exception:
        pass

    await cq.message.edit_text(
        f"❌ Оплату юзера {user_id} відхилено."
    )
    await cq.answer("❌ Відхилено")


# ── CALLBACK: tracker ──────────────────────
@router.callback_query(F.data == "tracker")
async def cb_tracker(cq: CallbackQuery):
    user = await get_user(cq.from_user.id)
    if not has_access(user, "premium"):
        await cq.answer(
            "🔒 Трекер доступний з тарифу Premium",
            show_alert=True
        )
        return

    today = datetime.now().strftime("%Y-%m-%d")
    async with aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row
        async with db.execute(
            "SELECT * FROM tracker WHERE user_id = ? AND date = ?",
            (cq.from_user.id, today)
        ) as cur:
            entries = await cur.fetchall()

    text = f"📅 <b>Трекер їжі — {today}</b>\n\n"
    if entries:
        for e in entries:
            text += f"• {e['meal']}: {e['note']}\n"
    else:
        text += "Записів немає. Надішли прийом їжі:\n\n"
        text += "Формат: <code>Сніданок: вівсянка, яйця, ягоди</code>"

    b = InlineKeyboardBuilder()
    b.button(text="🏠 Головне меню", callback_data="main_menu")
    await cq.message.edit_text(text, reply_markup=b.as_markup())
    await cq.answer()
```

```
# ── CALLBACK: plate_analysis ──────────────────
@router.callback_query(F.data == "plate_analysis")
async def cb_plate_analysis(cq: CallbackQuery):
    user = await get_user(cq.from_user.id)
    if not has_access(user, "premium"):
        await cq.answer(
            "🔒 Аналіз тарілки доступний з тарифу Premium",
            show_alert=True
        )
        return

    await cq.message.edit_text(
        "📸 <b>Аналіз тарілки</b>\n\n"
        "Надішли фото своєї тарілки і я проаналізую:\n"
        "• Наявність білку, жирів і вуглеводів\n"
        "• Частку овочів\n"
        "• Що варто додати або прибрати\n\n"
        "👇 Надішли фото прямо в чат",
        reply_markup=kb_back_main()
    )
    await cq.answer()


@router.message(F.content_type == ContentType.PHOTO)
async def handle_photo(msg: Message):
    user = await get_user(msg.from_user.id)
    if not has_access(user, "premium"):
        await msg.answer(
            "🔒 Аналіз фото тарілки доступний з тарифу Premium.\n"
            "Оновити тариф: /plans"
        )
        return

    await msg.answer(
        "🔍 Аналізую твою тарілку...\n\n"
        "📸 <b>Попередній аналіз:</b>\n\n"
        "✅ Є джерело білку\n"
        "✅ Є овочі\n"
        "⚠️ Рекомендую додати більше кольорових овочів\n"
        "⚠️ Джерело клітковини може бути більшим\n\n"
        "💡 <b>Порада:</b> додай ½ тарілки різнокольорових овочів\n"
        "і переконайся що є джерело корисних жирів (авокадо/олія)\n\n"
        "<i>Для детальнішого аналізу — VIP тариф з ШІ-нутриціологом</i>",
        reply_markup=kb_back_main()
    )
```

```
# ── CALLBACK: ai_nutritionist ─────────────────
@router.callback_query(F.data == "ai_nutritionist")
async def cb_ai_nutritionist(cq: CallbackQuery):
    user = await get_user(cq.from_user.id)
    if not has_access(user, "vip"):
        await cq.answer(
            "🔒 ШІ-нутриціолог доступний з тарифу VIP",
            show_alert=True
        )
        return

    await cq.message.edit_text(
        "🤖 <b>ШІ-нутриціолог</b>\n\n"
        "Я готовий відповісти на будь-яке питання про харчування.\n\n"
        "Питай прямо в чат, наприклад:\n"
        "• «Що краще їсти після тренування?»\n"
        "• «Як підвищити споживання білку?»\n"
        "• «Чому я хочу солодке щовечора?»\n"
        "• «Яка норма вітаміну D?»\n\n"
        "💬 Напиши своє питання 👇",
        reply_markup=kb_back_main()
    )
    await cq.answer()


# ── CALLBACK: my_profile ──────────────────────
@router.callback_query(F.data == "my_profile")
async def cb_my_profile(cq: CallbackQuery):
    user = await get_user(cq.from_user.id)
    if not user:
        await cq.answer("⚠️ Профіль не знайдено", show_alert=True)
        return

    plan_name = PLANS.get(user["plan"], {}).get("name", "Free") if user["plan"] !
    until_str = user.get("plan_until", "")
    if until_str:
        until = datetime.strptime(until_str, "%Y-%m-%d %H:%M:%S")
        days_left = max(0, (until - datetime.now()).days)
        until_label = f"{until.strftime('%d.%m.%Y')} (ще {days_left} дн.)"
    else:
        until_label = "—"

    text = (
        f"👤 <b>Мій профіль</b>\n\n"
        f"🆔 ID: <code>{user['user_id']}</code>\n"
        f"📛 Ім'я: {user.get('full_name','')}\n"
        f"💎 Тариф: {plan_name}\n"
```

```
        f"📅 Діє до: {until_label}\n"
        f"📚 Тиждень контенту: {user.get('week',1)}/28\n\n"
    )

    if user.get("calories"):
        goal_label = GOALS.get(user["goal"], ("—",))[0]
        text += (
            f"<b>📊 Мій КБЖУ:</b>\n"
            f"🔥 {user['calories']} ккал/день\n"
            f"🥩 Білок: {user['protein']}г\n"
            f"🧈 Жири: {user['fat']}г\n"
            f"🌾 Вуглеводи: {user['carbs']}г\n"
            f"🎯 Ціль: {goal_label}\n"
        )

    b = InlineKeyboardBuilder()
    b.button(text="🔄 Перерахувати КБЖУ", callback_data="kbju_start")
    b.button(text="💎 Продовжити підписку", callback_data="plans")
    b.button(text="🏠 Головне меню",        callback_data="main_menu")
    b.adjust(1)

    await cq.message.edit_text(text, reply_markup=b.as_markup())
    await cq.answer()


# ────────────────────────────────────────────────
# АДМІН ПАНЕЛЬ
# ────────────────────────────────────────────────
def is_admin(user_id: int) -> bool:
    return user_id == ADMIN_ID


@router.message(Command("admin"))
async def cmd_admin(msg: Message):
    if not is_admin(msg.from_user.id):
        return
    await msg.answer(
        "🔐 <b>Адмін панель NutriSense</b>",
        reply_markup=kb_admin()
    )


@router.callback_query(F.data == "adm_stats")
async def adm_stats(cq: CallbackQuery):
    if not is_admin(cq.from_user.id):
        return
    stats = await get_stats()
```

```
    text = (
        f"📊 <b>Статистика NutriSense</b>\n\n"
        f"👥 Всього юзерів: {stats['total']}\n"
        f"💎 Платних: {stats['paid']}\n"
        f"🆓 Безкоштовних: {stats['total'] - stats['paid']}\n"
        f"✅ Успішних оплат: {stats['payments']}\n"
        f"💰 Загальний дохід: {stats['revenue']} ₴\n"
        f"📅 Дата: {datetime.now().strftime('%d.%m.%Y %H:%M')}"
    )
    b = InlineKeyboardBuilder()
    b.button(text="◀️ Назад", callback_data="adm_back")
    await cq.message.edit_text(text, reply_markup=b.as_markup())
    await cq.answer()


@router.callback_query(F.data == "adm_users")
async def adm_users(cq: CallbackQuery):
    if not is_admin(cq.from_user.id):
        return
    users = await get_all_users()
    text = f"👥 <b>Користувачі ({len(users)}):</b>\n\n"
    for u in users[:20]:
        plan = u.get("plan", "free")
        name = u.get("full_name") or u.get("username") or str(u["user_id"])
        text += f"• {name} | {plan} | тиж.{u.get('week',1)}\n"
    if len(users) > 20:
        text += f"\n<i>...і ще {len(users)-20}</i>"

    b = InlineKeyboardBuilder()
    b.button(text="◀️ Назад", callback_data="adm_back")
    await cq.message.edit_text(text, reply_markup=b.as_markup())
    await cq.answer()


@router.callback_query(F.data == "adm_broadcast")
async def adm_broadcast(cq: CallbackQuery, state: FSMContext):
    if not is_admin(cq.from_user.id):
        return
    await state.set_state(AdminBroadcastState.waiting_text)
    await cq.message.edit_text(
        "📢 <b>Розсилка</b>\n\nНадішли текст для розсилки всім юзерам.\n"
        "Підтримується HTML форматування."
    )
    await cq.answer()


@router.message(AdminBroadcastState.waiting_text)
```

```
async def adm_do_broadcast(msg: Message, state: FSMContext):
    if not is_admin(msg.from_user.id):
        return
    await state.clear()
    users = await get_all_users()
    sent = failed = 0
    for u in users:
        try:
            await msg.bot.send_message(u["user_id"], msg.text)
            sent += 1
        except Exception:
            failed += 1
    await msg.answer(
        f"✅ Розсилка завершена\n"
        f"📤 Відправлено: {sent}\n"
        f"❌ Помилок: {failed}",
        reply_markup=kb_admin()
    )


@router.callback_query(F.data == "adm_confirm_pay")
async def adm_confirm_pay(cq: CallbackQuery, state: FSMContext):
    if not is_admin(cq.from_user.id):
        return
    await state.set_state(AdminPaymentState.waiting_user_id)
    await cq.message.edit_text(
        "✅ <b>Ручне підтвердження оплати</b>\n\n"
        "Введи Telegram ID юзера:"
    )
    await cq.answer()


@router.message(AdminPaymentState.waiting_user_id)
async def adm_pay_user_id(msg: Message, state: FSMContext):
    if not is_admin(msg.from_user.id):
        return
    try:
        uid = int(msg.text.strip())
        await state.update_data(target_user_id=uid)
        await state.set_state(AdminPaymentState.waiting_plan)

        b = InlineKeyboardBuilder()
        for key in PLANS:
            b.button(text=PLANS[key]["name"], callback_data=f"adm_setplan_{uid}_{
        b.adjust(1)
        await msg.answer("Обери тариф:", reply_markup=b.as_markup())
    except ValueError:
```

```
        await msg.answer("⚠️ Невірний ID. Введи число:")


@router.callback_query(F.data.startswith("adm_setplan_"))
async def adm_set_plan(cq: CallbackQuery, state: FSMContext):
    if not is_admin(cq.from_user.id):
        return
    await state.clear()
    parts = cq.data.split("_")
    uid, plan = int(parts[2]), parts[3]

    plan_until = (
        datetime.now() + timedelta(days=PLANS[plan]["days"])
    ).strftime("%Y-%m-%d %H:%M:%S")

    await upsert_user(uid, plan=plan, plan_until=plan_until)

    # Повідомлення юзеру
    try:
        await cq.bot.send_message(
            uid,
            f"🎉 <b>Підписка активована!</b>\n\n"
            f"Тариф: {PLANS[plan]['name']}\n"
            f"Діє до: {plan_until[:10]}\n\n"
            f"Дякуємо за довіру! 🌿\n"
            f"/menu — перейти до головного меню"
        )
    except Exception:
        pass

    await cq.message.edit_text(
        f"✅ Тариф {PLANS[plan]['name']} активовано для юзера {uid}",
        reply_markup=kb_admin()
    )
    await cq.answer()


@router.callback_query(F.data == "adm_send_content")
async def adm_send_content(cq: CallbackQuery):
    if not is_admin(cq.from_user.id):
        return
    await cq.answer("⏳ Відправляю контент...")

    users = await get_all_users()
    sent = 0
    for u in users:
        if not has_access(u, "start"):
```

```
            continue
        week_num = u.get("week", 1) or 1
        week_data = WEEKLY_CONTENT.get(week_num)
        if not week_data:
            continue

        # Відправляємо перший пост тижня
        posts = week_data["posts"]
        if not posts:
            continue

        post = posts[0]
        try:
            b = InlineKeyboardBuilder()
            b.button(
                text="📚 Всі матеріали тижня",
                callback_data="weekly_content"
            )
            await cq.bot.send_message(
                u["user_id"], post["text"],
                reply_markup=b.as_markup()
            )
            # Переводимо на наступний тиждень (max 28)
            next_week = min(week_num + 1, 28)
            await upsert_user(u["user_id"], week=next_week)
            sent += 1
        except Exception:
            pass

    await cq.message.edit_text(
        f"✅ Контент відправлено {sent} юзерам",
        reply_markup=kb_admin()
    )


@router.callback_query(F.data == "adm_back")
async def adm_back(cq: CallbackQuery):
    if not is_admin(cq.from_user.id):
        return
    await cq.message.edit_text(
        "🔐 <b>Адмін панель NutriSense</b>",
        reply_markup=kb_admin()
    )
    await cq.answer()


# ── MONOBANK WEBHOOK (якщо є WEBHOOK_URL) ────
```

```
async def handle_mono_webhook(request):
    """Обробник webhook від Monobank"""
    from aiohttp.web import Request, Response
    try:
        data = await request.json()
        invoice_id = data.get("invoiceId")
        status     = data.get("status")
        if invoice_id and status == "success":
            result = await confirm_payment(invoice_id)
            if result:
                user_id, plan = result
                # Повідомлення юзеру
                bot = request.app["bot"]
                await bot.send_message(
                    user_id,
                    f"🎉 <b>Оплату підтверджено!</b>\n\n"
                    f"Тариф: {PLANS[plan]['name']}\n"
                    f"Підписка активна на 30 днів 🌿\n\n"
                    f"/menu — головне меню"
                )
                # Повідомлення адміну
                await bot.send_message(
                    ADMIN_ID,
                    f"💰 <b>Нова оплата!</b>\n"
                    f"Юзер: {user_id}\n"
                    f"Тариф: {plan}\n"
                    f"Сума: {PLANS[plan]['price_uah']} ₴"
                )
    except Exception as e:
        log.error(f"Webhook error: {e}")
    return Response(text="ok")


# ────────────────────────────────────────────────
# SCHEDULER: автоматична розсилка контенту
# ────────────────────────────────────────────────
async def schedule_weekly_content(bot: Bot):
    """
    Кожного понеділка о 10:00 відправляє перший пост тижня
    всім платним підписникам і збільшує лічильник тижня.
    """
    while True:
        now = datetime.now()
        # Понеділок = weekday 0, 10:00
        days_until_monday = (7 - now.weekday()) % 7 or 7
        next_run = now.replace(hour=10, minute=0, second=0, microsecond=0)
        next_run += timedelta(days=days_until_monday)
```

```
        wait_seconds = (next_run - now).total_seconds()

        log.info(f"⏰ Наступна розсилка: {next_run.strftime('%d.%m.%Y %H:%M')}")
        await asyncio.sleep(wait_seconds)

        users = await get_all_users()
        sent = 0
        for u in users:
            if not has_access(u, "start"):
                continue
            week_num = u.get("week", 1) or 1
            week_data = WEEKLY_CONTENT.get(week_num)
            if not week_data:
                continue
            posts = week_data["posts"]
            if not posts:
                continue
            post = posts[0]
            try:
                b = InlineKeyboardBuilder()
                b.button(
                    text="📚 Всі матеріали тижня",
                    callback_data="weekly_content"
                )
                await bot.send_message(
                    u["user_id"],
                    f"📚 <b>Новий тиждень починається!</b>\n\n"
                    f"<b>Тиждень {week_num}: {week_data['theme']}</b>\n\n"
                    + post["text"],
                    reply_markup=b.as_markup()
                )
                next_week = min(week_num + 1, 28)
                await upsert_user(u["user_id"], week=next_week)
                sent += 1
            except Exception as e:
                log.warning(f"Cannot send to {u['user_id']}: {e}")

        log.info(f"✅ Розсилка завершена: {sent} юзерів")


# ────────────────────────────────────────────
# MAIN
# ────────────────────────────────────────────
async def main():
    await init_db()

    bot = Bot(
```

```
        token=BOT_TOKEN,
        default=DefaultBotProperties(parse_mode=ParseMode.HTML)
    )
    dp = Dispatcher(storage=MemoryStorage())
    dp.include_router(router)

    # Запуск планувальника в фоні
    asyncio.create_task(schedule_weekly_content(bot))

    log.info("🚀 NutriSense Bot запущено")

    if WEBHOOK_URL:
        # WEBHOOK режим
        from aiohttp import web
        app = web.Application()
        app["bot"] = bot
        app.router.add_post("/webhook", dp.feed_webhook_update)
        app.router.add_post("/mono_webhook", handle_mono_webhook)

        await bot.set_webhook(f"{WEBHOOK_URL}/webhook")
        runner = web.AppRunner(app)
        await runner.setup()
        site = web.TCPSite(runner, "0.0.0.0", 8080)
        await site.start()
        log.info(f"🌐 Webhook запущено на порту 8080")
        await asyncio.Event().wait()
    else:
        # POLLING режим
        await bot.delete_webhook(drop_pending_updates=True)
        await dp.start_polling(bot)


if __name__ == "__main__":
    asyncio.run(main())
```